

REBUILT Trajectory Calculations

Anthony Le

February 14, 2026

Calculations for launching fuel projectiles for the 2026 FIRST Robotics Competition: REBUILT.

Goal

The goal is for the launcher to automatically aim to launch fuel at a target, without any human operation. Calculate the unknowns from the given parameters with as close to a fully closed-form (mathematically exact) solution as possible.

Math Requirements

All of the calculations heavily use vector algebra. The properties of vectors are taught in one of the below sections.

Along with vectors, knowledge of trigonometric and inverse trigonometric functions is important because of the presence of angles.

Decent physics skills are needed to derive many of the equations.

For the last part of calculations involving rotations, which is more separate and not important to the trajectory calculations, matrix-like transformations are needed, but all definitions will be listed.

Topics:

- Trigonometry, inverse trigonometry
- Vector algebra
- Algebra
- Physics, kinematics

Difficulty: at least *AP Physics C: Mechanics* level.

Math Preface

These calculations aim to use minimal intuition and simple operations, for accessibility.

Try not to be intimidated by the notation and long rows of calculations.

These calculations are mostly just algebraic manipulations of an initial equation derived from physics.

There are not many difficult concepts or complex operations; instead, simple equations are repeatedly transformed and altered with algebraic operations to reach a final useful result.

Fancy tricks, esoteric or technical concepts, and profound or clever insights are avoided.

The general structure of each section of math is this: 1. Define variables: givens, unknowns, useful definitions 2. Use simple intuition or physics to create an equation with the variables 3. Perform many algebraic operations on the equation one-by-one to change it into a more useful equation 4. The final equation reveals new information, such as an expression for an unknown

Background

This section will teach you the definitions and the more advanced math topics that are needed to understand the calculations.

Terminology and Conventions

We are launching at a *target*. The target is either the *hub* when the robot is inside the alliance zone, or a *zone* via the trench, bump, or over the hub.

The *trajectory* of the *projectile* (fuel) starts at the *muzzle* (the point in space where the projectile exits the launcher) and ends at the target. For the duration of the trajectory, the projectile is only influenced by gravity (it is ‘ballistic’).

For pitch angle, an angle of 0 is parallel to the ground.

A yaw (turn) angle of 0 is facing forward.

Reference Frames and Coordinate Systems

A *reference frame* is like a perspective.

Imagine two cars moving in the same forward direction, Car A at 30 mph and Car B at 40 mph. From the reference frame (perspective) of Car A, Car B is moving forward away at 10mph. From the reference frame of Car B, Car A is moving backwards at 10 mph.

Coordinate systems are used to define reference frames. They are defined with an origin (where the point $\langle 0, 0, 0 \rangle$ is), a rotation (which way the X and Y axes face), and a velocity (how the entire system, or origin, is moving).

The z axis in 3D space is the vertical (positive is upwards) direction. The x and y axes are horizontal. All axes are perpendicular to each other.

The default coordinate system used for vectors is the field coordinate system (blue origin), abbreviated **FCS**. Just think of it as an unchanging, static, or absolute system.

The robot coordinate system, abbreviated **RCS**, is relative to the robot. Vectors with this coordinate system are “attached” to or follow the robot’s reference frame; its position and rotation. An example is the locations of the robot wheels relative to the robot.

Syntax

Greek Letters

Names of Greek letters that will be used as variables.

- α : alpha (lowercase)
- θ : theta (lowercase)
- ϕ : phi (lowercase)
- σ : sigma (lowercase)
- ω : omega (lowercase)

Symbols

The $:=$ (colon equals) symbol means assignment or definition, “is defined as”; true by definition, rather than a result or equality. (It has the same effect as $=$.)

$$x := (\text{distance})$$

The $\implies$ arrow means “implies”, “it follows”; deduction, algebra.

$$x = y^3 \implies y = \sqrt[3]{x}$$

$\Longleftrightarrow$ is the same but it implies in both directions. It may be used when the reverse deduction is necessary or emphasized.

The $\hookrightarrow$ arrow shall mean that the equation is derived from intuition or physics, rather than derived algebraically from another equation.

$$\hookrightarrow F_{net} = F_T - mg \sin \theta$$

$\in$ means “in” or “is a member of” and is used to say what set of numbers a value is in.
 $\mathbb{R}$ is the set of all real numbers. $\mathbb{R}^+$ is the set of all positive real numbers.

$$t \in \mathbb{R}^+ \Longleftrightarrow (t > 0 \text{ and } t \text{ is real})$$

$\text{atan2}(y, x)$ is a variant of $\arctan(y/x)$ that takes two separate numbers. The difference is that atan2 can give angles in any of the four quadrants of the 2D plane, while $\arctan$ can only give angles in the rightmost two quadrants.

Vectors

A *vector* is like a list of numbers called components. Each component corresponds to a coordinate in space. 2D vectors have x and y coordinate components, 3D vectors additionally have a z component.

A regular number is called a *scalar*.

Vector variables have an arrow on top, e.g. $\vec{v}$, and have components v_x, v_y , and optionally v_z .

Vectors can be directly written out with their components using angled brackets: $\vec{v}_{xyz} = \langle v_x, v_y, v_z \rangle$.

For a 3D vector $\vec{v}_{xyz}$, define $\vec{v}_{xy}$ as the 2D horizontal vector of it.

A *unit vector* is a vector with magnitude 1. They have a hat on top instead of an arrow, e.g. $\hat{w}$.

Basic vector arithmetic:

$$\begin{aligned} c\vec{A} &= \vec{Ac} = \langle cA_x, cA_y, \dots \rangle \\ \vec{A} \pm \vec{B} &= \langle A_x \pm B_x, A_y \pm B_y, \dots \rangle \end{aligned}$$

More vector properties and operations:

$$\begin{aligned} \|\vec{A}\| &:= (\text{magnitude of vector}) = \sqrt{A_x^2 + A_y^2 + \dots} \\ \angle \vec{A}_{xy} &:= (\text{angle } \vec{A} \text{ makes with } x\text{-axis}) = \text{atan2}(A_y, A_x) \\ \vec{0} &:= (\text{zero vector}) = \langle 0, 0, \dots \rangle \\ \vec{A} \cdot \vec{B} &:= (\text{dot product}) = A_x B_x + A_y B_y + \dots = \|\vec{A}\| \|\vec{B}\| \cos \theta \\ &\implies \vec{A} \cdot \vec{A} = A_x^2 + A_y^2 + \dots = \|\vec{A}\|^2 \\ \vec{A}_{xy} \times \vec{B}_{xy} &:= (\text{2D scalar cross product}) = A_x B_y - A_y B_x = \|\vec{A}\| \|\vec{B}\| \sin \theta \\ \vec{A} = \vec{B} &\implies \|\vec{A}\| = \|\vec{B}\| \\ \vec{A}_{xy} &= \|\vec{A}\| \langle \cos \angle \vec{A}, \sin \angle \vec{A} \rangle \end{aligned}$$

Rotations

This section can be skipped if necessary as these skills aren't important in the primary trajectory calculations.

An $\mathbf{R}$ denotes a rotation (either 2D or 3D).

Internally and mathematically, they are represented either as transformation matrices or quaternions.

These can transform vectors and other rotations via multiplication. The rotation $\mathbf{R}$ is *applied* to the other object.

$$\begin{aligned}\mathbf{R}\vec{v} &= \vec{v}' \\ \mathbf{R}_2\mathbf{R}_1 &= \mathbf{R}'_1\end{aligned}$$

$\mathbf{R}(a)$ is a 2D rotation with angle a .

$\mathbf{R}(a, b, c)$ is a 3D rotation with roll a , pitch b , yaw c .

The following functions will be defined to access the individual angles:

$$\text{roll}(\mathbf{R}) = a, \text{pitch}(\mathbf{R}) = b, \text{yaw}(\mathbf{R}) = c$$

(Yaw is turning side to side, pitch is turning up and down, roll is rotating around while keeping same direction.)

Properties

Forward Vector The *forward vector* of $\mathbf{R}$ is a unit vector pointing in the direction that an object with rotation $\mathbf{R}$ is facing.

Objects that aren't rotated face in the $+x$ axis.

Let:

$$\begin{aligned}\hat{i} &:= (\text{unit forward vector for } \mathbf{R}(0, 0, 0)) = \langle 1, 0, 0 \rangle \\ \mathbf{R}\hat{i} &= (\text{unit vector for } \mathbf{R})\end{aligned}$$

Forward vector for 2D rotation:

$$\mathbf{R}(a)\hat{i} = \langle \cos a, \sin a \rangle$$

Forward vector for 3D rotation:

$$\mathbf{R}(a, b, c)\hat{i} = \langle \cos b \cos c, \cos b \sin c, -\sin b \rangle$$

Notice that there is no roll a in the forward vector: this is because roll simply rotates the vector around in-place as if the vector arrow tip was a propeller, without changing its direction.

Pitch Direction Note that the z component is negative (or equivalently, c is negative). This is because the coordinate system is implicitly North-West-Up (NWU) axes convention, so using the right hand rule on pitch results in positive pitch tilting downwards.

For the purposes of almost all calculations, this will be ignored for convenience and positive pitch will be considered to be upwards. But when using 3D rotations, the pitch should follow correct conventions, so a given external pitch must be negated.

Algebra This algebra applies mainly to 3D rotations since 2D rotation algebra is trivial.

Let $\mathbf{I}$ be the identity rotation; no rotation.

Then $\mathbf{I}\vec{v} = \vec{v}$ and $\mathbf{I}\mathbf{R} = \mathbf{R}$.

Let $\mathbf{R}^{-1}$ be the inverse rotation of $\mathbf{R}$.

Then, $\mathbf{R}^{-1}\mathbf{R} = \mathbf{R}\mathbf{R}^{-1} = \mathbf{I}$.

Rotation multiplication follows the associative property:

$$\begin{aligned}(\mathbf{R}_1\mathbf{R}_2)\mathbf{R}_3 &= \mathbf{R}_1(\mathbf{R}_2\mathbf{R}_3) \\ (\mathbf{R}_1\mathbf{R}_2)\vec{v} &= \mathbf{R}_1(\mathbf{R}_2\vec{v})\end{aligned}$$

3D rotations do NOT follow the commutative property in general:

$$\mathbf{R}_1 \mathbf{R}_2 \neq \mathbf{R}_2 \mathbf{R}_1$$

Kinematics Equations

We will use simple equations that describe how position, velocity, and acceleration are related.

For constant velocity:

$$x = vt$$
$$v = \frac{x}{t}$$

For constant acceleration:

$$v = at$$
$$a = \frac{v}{t}$$

Kinematics motion equation, for constant acceleration, where v_0 is initial velocity:

$$x = v_0 t + \frac{1}{2} a t^2$$

Variables

This specific projectile launch problem will now be described.

Below are all of the directly given or unknown variables.

Try to remember all of the variable symbols.

Givens

These are the given (always known) variables that are used to calculate the unknowns.

$$\begin{aligned}\vec{P}_{xyz} &:= (\text{robot position}) \\ \vec{V}_{xy} &:= (\text{robot velocity}) \\ \mathbf{R}_R &:= (\text{robot 3D rotation}) \\ \vec{T}_{xyz} &:= (\text{target position}) \\ \vec{Q}_{xyz} &:= (\text{launcher position, RCS}) \\ g &:= (\text{gravitational acceleration constant}) \approx 9.8m/s^2\end{aligned}$$

Unknowns

These are the unknown (controllable) variables that must be determined.

$$\begin{aligned}\theta &:= (\text{launcher pitch, FCS}) \\ \alpha &:= (\text{launcher yaw, FCS}) \\ u &:= (\text{projectile exit speed})\end{aligned}$$

The projectile exit speed is actually a function of both the flywheel angular speed and the launcher pitch, due to the launching mechanism. Define U as this function.

$$\begin{aligned}\omega &:= (\text{flywheel angular speed}) \\ u &:= U(\omega, \theta)\end{aligned}$$

Additionally, time of flight is an important derived unknown that appears in the calculations.

$$t := (\text{time of flight})$$

The launch angles must be converted to RCS for the final result.

$$\begin{aligned}\theta_r &:= (\text{launcher pitch, RCS}) \\ \alpha_r &:= (\text{launcher yaw, RCS})\end{aligned}$$

The two “core” unknowns are θ and u , while α and t are usually derived unknowns. (α_r and θ_r are directly derived.)

Notes

The speed of the target position shall always be 0; none of our targets are moving objects whose velocity needs to be accounted for.

The launcher has 0 roll; it cannot roll.

The launcher pitch θ must be between 0 and $\frac{\pi}{2}$.

Projectile Exit Speed

Due to the arbitrary ω , the dependency on θ can be eliminated by choosing ω appropriately, to counter the impact of θ .

The function U must be modeled empirically using a large, varied data set.

There is likely noticeable natural variation in U , even with the best model.

Note that the launching mechanism also has backrollers that run at a constant and customizable ratio to the flywheel.

Robot Velocity

The robot velocity $\vec{V}$ is necessary to know because the robot’s velocity contributes to the fuel’s velocity.

As an example, imagine dropping a heavy object from a moving plane versus from a floating helicopter right when it is directly above a target on the ground. The object dropped from the plane will miss because it has forward velocity inherited from the plane.

Also, note that if this was not considered and $\vec{V} = \vec{0}$, then all of the trajectory calculations would be trivial.

Setup

Set up some common and useful definitions for convenience.

Displacement

Many of the given parameters are primarily just used to calculate the known displacement (muzzle) from the projectile exit point to the target.

$$\vec{S}_{xyz} := (\text{target displacement})$$

$$\vec{M}_{xyz} := (\text{muzzle position, FCS}) = \vec{P} + \mathbf{R}_R \vec{Q}$$

$$\vec{S} = \vec{T} - \vec{M}$$

$$(\text{substitute for } \vec{M}) \implies \vec{S} = \vec{T} - (\vec{P} + \mathbf{R}_R \vec{Q})$$

(Note that there are operations available in code for applying the robot pose to the launcher position as a change of reference frame.)

Exit Velocity

The initial exit velocity $\vec{v}$ is the launch vector with magnitude u at angles θ and α , plus the contributing robot velocity. It is unknown.

Also define $\hat{w}$ as the unit launch direction (ignoring $\vec{V}$), which is the forward vector of the rotation from the launch angles.

$$\hat{w} := (\text{unit launch direction, FCS}) := \mathbf{R}(0, -\theta, \alpha) \hat{i}$$

$$(\text{forward vector}) \implies \hat{w} = \langle \cos \theta \cos \alpha, \cos \theta \sin \alpha, \sin \theta \rangle$$

$$\vec{v}_{xyz} := (\text{projectile exit velocity at muzzle, FCS}) = u\hat{w} + \vec{V}$$

$$v_x = u \cos \theta \cos \alpha + V_x$$

$$v_y = u \cos \theta \sin \alpha + V_y$$

$$v_z = u \sin \theta$$

Gravity Vector

For convenience, let $\vec{g}$ be the gravitational acceleration vector. Also notice that this is the only acceleration acting on the projectile during the trajectory.

Note the *negative* component.

$$\vec{g} := \langle 0, 0, -g \rangle$$

Horizontal Effective Displacement

Define $\vec{L}$ as the horizontal (x and y directions only) “effective displacement” vector, dependent on t .

This comes from hypothetically changing the reference frame from being still to having velocity $\vec{V}$.

The situation where the robot moves while the target is still is **perfectly analogous** to the situation where the robot is still while the target moves (with opposite velocity).

It is like ‘subtracting $\vec{V}$ from both sides.

Now, the situation is like we are attempting to launch a moving target with velocity $-\vec{V}$, from a static launcher, so we must *predict the target’s future position*.

So, define the very useful vector of the effective horizontal displacement:

$$\vec{L}_{xy} := (\text{horizontal effective displacement})$$

$$\vec{L} = \vec{S}_{xy} - \vec{V}t = \langle S_x - V_x t, S_y - V_y t \rangle$$

Approaches

Due to the multiple unknowns, there is a large solution space for getting the projectile from the muzzle to the target; either zero or infinitely many solutions. Some are better or more appropriate than others. Also, due to physical limitations, only a subset of these theoretically possible solutions are valid for our robot.

So we need a way to specifically pick excellent, valid solutions from this space. Choosing and locking a value for one of the unknowns achieves this.

Exactly one of unknowns must be made into a known parameter by choosing or guessing a value for it (usually multiple) before any calculations are done.

Guess less unknowns, and the solution space is too large. More, and the described solution is basically pre-chosen already without calculations.

α and t do not make much sense as a parameter since they are more easily derived from the other unknowns, and it is harder to guess a good and valid trajectory with these.

Therefore, there are only two reasonable approaches:

- Make θ (launch pitch) a parameter
- Make u (exit speed) a parameter

Common Calculations

Some methods or calculations are the same in both approaches.

Also, in both approaches, the main strategy is to find the finite set of valid t solutions and then use t to find the unknowns.

Calculating α from t

α can be directly calculated if t is known.

Start with the definition of constant velocity for the horizontal direction:

$$\hookrightarrow \vec{v}_{xy} = \frac{\vec{S}_{xy}}{t}$$

$$(\text{substitute for } \vec{v}_{xy}) \implies \langle u \cos \theta \cos \alpha + V_x, u \cos \theta \sin \alpha + V_y \rangle = \frac{\vec{S}_{xy}}{t}$$

$$(\text{subtract } \vec{V} \text{ from sides}) \implies \langle u \cos \theta \cos \alpha, u \cos \theta \sin \alpha \rangle = \frac{\vec{S}_{xy}}{t} - \vec{V}$$

$$(\text{multiply sides by } t) \implies \langle ut \cos \theta \cos \alpha, ut \cos \theta \sin \alpha \rangle = \vec{S}_{xy} - \vec{V}t$$

$$(\text{substitute } \vec{L} \text{ in}) \implies \langle ut \cos \theta \cos \alpha, ut \cos \theta \sin \alpha \rangle = \vec{L}$$

Now, take the angle of the vector on each side.

$$(\text{take angle of sides}) \implies \angle \langle ut \cos \theta \cos \alpha, ut \cos \theta \sin \alpha \rangle = \angle \vec{L}$$

$$(\text{use definition of } \angle) \implies \arctan\left(\frac{ut \cos \theta \sin \alpha}{ut \cos \theta \cos \alpha}\right) = \angle \vec{L}$$

$$(\text{simplify fraction}) \implies \arctan(\tan \alpha) = \angle \vec{L}$$

$$(\text{inverse functions cancel}) \implies \boxed{\alpha = \angle \vec{L}}$$

Intuitively: if we choose the reference frame so that the launcher is static, then the launch angle is simply aiming at the future position of the target.

Equation of u , θ , and t .

An equation containing u , θ , and t is useful in both approaches to either eliminate one of the unknowns or easily solve for one of them.

The most efficient way to find this is to use the horizontal velocity which has no acceleration.

Start with one of the intermediate equations derived just earlier for calculating α with t , before the step of taking the angle.

$$\langle ut \cos \theta \cos \alpha, ut \cos \theta \sin \alpha \rangle = \vec{L}$$

Instead of taking the angle of the vector on each side, take the magnitude.

$$(\text{take magnitude of sides}) \implies \|\langle ut \cos \theta \cos \alpha, ut \cos \theta \sin \alpha \rangle\| = \|\vec{L}\|$$

$$(\text{factor out scalars}) \implies (ut \cos \theta) \|\langle \cos \alpha, \sin \alpha \rangle\| = \|\vec{L}\|$$

$$(\text{unit vector: } \|\langle \cos \alpha, \sin \alpha \rangle\| = 1) \implies ut \cos \theta = \|\vec{L}\|$$

$$(\text{divide sides by } t \cos \theta) \implies \boxed{u = \frac{\|\vec{L}\|}{t \cos \theta}}$$

Intuitively: if we choose the reference frame so that the launcher is static, then the exit speed multiplied by time is equal to the total horizontal distance to the future target.

θ Parameter Approach

Suppose θ is a parameter for the purposes of our calculations.

1. Find t

Derive Equation for t

Consider the vertical direction only (z component). Start with the kinematics equation:

$$\hookrightarrow S_z = v_z t - \frac{1}{2} g t^2$$

Recall three definitions from earlier:

$$v_z = u \sin \theta$$

$$u = \frac{\|\vec{L}\|}{t \cos \theta}$$

$$\vec{L} = \vec{S}_{xy} - \vec{V} t$$

Substitute:

$$(\text{substitute for } v_z) \implies S_z = (u \sin \theta) t - \frac{1}{2} g t^2$$

$$(\text{substitute for } u) \implies S_z = \left(\frac{\|\vec{L}\|}{t \cos \theta} \right) (\sin \theta) t - \frac{1}{2} g t^2$$

$$(\text{substitute for } \vec{L}) \implies S_z = \left(\frac{\|\vec{S}_{xy} - \vec{V} t\|}{t \cos \theta} \right) (\sin \theta) t - \frac{1}{2} g t^2$$

We now have an equation with only one unknown, t . Simplify the equation:

$$\begin{aligned} \text{(rearrange fraction)} &\implies S_z = \|\vec{S}_{xy} - \vec{V}t\| \frac{t \sin \theta}{t \cos \theta} - \frac{1}{2}gt^2 \\ \text{(simplify fraction)} &\implies S_z = \|\vec{S}_{xy} - \vec{V}t\| \tan \theta - \frac{1}{2}gt^2 \end{aligned}$$

Solve for Vector Magnitude

Solve for the vector magnitude expression before continuing:

$$\begin{aligned} \vec{S}_{xy} - \vec{V}t &= \langle S_x - V_x t, S_y - V_y t \rangle \\ \text{(magnitude formula)} &\implies \|\vec{S}_{xy} - \vec{V}t\| = \sqrt{(S_x - V_x t)^2 + (S_y - V_y t)^2} \\ \text{(apply squaring)} &\implies \|\vec{S}_{xy} - \vec{V}t\| = \sqrt{(S_x^2 - 2S_x V_x t + V_x^2 t^2) + (S_y^2 - 2S_y V_y t + V_y^2 t^2)} \\ \text{(rearrange terms)} &\implies \|\vec{S}_{xy} - \vec{V}t\| = \sqrt{S_x^2 + S_y^2 - 2S_x V_x t - 2S_y V_y t + V_x^2 t^2 + V_y^2 t^2} \\ \text{(factor out each } t \text{ power)} &\implies \|\vec{S}_{xy} - \vec{V}t\| = \sqrt{(S_x^2 + S_y^2) - 2(S_x V_x + S_y V_y)t + (V_x^2 + V_y^2)t^2} \end{aligned}$$

By vector properties:

$$\begin{aligned} V_x^2 + V_y^2 &= \|\vec{V}\|^2 \\ S_x^2 + S_y^2 &= \|\vec{S}_{xy}\|^2 \\ S_x V_x + S_y V_y &= S_x V_x + S_y V_y + (\vec{S} \cdot \vec{0}) = \vec{S} \cdot \vec{V} \end{aligned}$$

$$\text{(substitute)} \implies \|\vec{S}_{xy} - \vec{V}t\| = \sqrt{\|\vec{S}_{xy}\|^2 - 2(\vec{S} \cdot \vec{V})t + \|\vec{V}\|^2 t^2}$$

Convert t Equation to Quartic

Now, substate for the vector magnitude in the original equation:

$$\begin{aligned} \text{(substitute for magnitude)} &\implies S_z = (\sqrt{\|\vec{S}_{xy}\|^2 - 2(\vec{S} \cdot \vec{V})t + \|\vec{V}\|^2 t^2})(\tan \theta) - \frac{1}{2}gt^2 \\ \text{(add } \frac{1}{2}gt^2 \text{ to sides)} &\implies S_z + \frac{1}{2}gt^2 = (\sqrt{\|\vec{S}_{xy}\|^2 - 2(\vec{S} \cdot \vec{V})t + \|\vec{V}\|^2 t^2})(\tan \theta) \\ \text{(square sides)} &\implies (S_z + \frac{1}{2}gt^2)^2 = ((\sqrt{\|\vec{S}_{xy}\|^2 - 2(\vec{S} \cdot \vec{V})t + \|\vec{V}\|^2 t^2})(\tan \theta))^2 \\ \text{(apply squaring)} &\implies S_z^2 + S_z gt^2 + \frac{1}{4}g^2 t^4 = (\|\vec{S}_{xy}\|^2 - 2(\vec{S} \cdot \vec{V})t + \|\vec{V}\|^2 t^2)(\tan^2 \theta) \end{aligned}$$

For conciseness, define a symbol for $\tan^2 \theta$, a known constant.

$$M := \tan^2 \theta$$

$$\begin{aligned}
(\text{substitute } M \text{ in}) &\implies S_z^2 + S_z g t^2 + \frac{1}{4} g^2 t^4 = (\|\vec{S}_{xy}\|^2 - 2(\vec{S} \cdot \vec{V})t + \|\vec{V}\|^2 t^2) M \\
(\text{distribute } M) &\implies S_z^2 + S_z g t^2 + \frac{1}{4} g^2 t^4 = M \|\vec{S}_{xy}\|^2 - 2M(\vec{S} \cdot \vec{V})t + M \|\vec{V}\|^2 t^2 \\
(\text{rearrange terms}) &\implies \frac{1}{4} g^2 t^4 + S_z g t^2 - M \|\vec{V}\|^2 t^2 + 2M(\vec{S} \cdot \vec{V})t + S_z^2 - M \|\vec{S}_{xy}\|^2 = 0 \\
(\text{factor out each } t \text{ power}) &\implies \boxed{\left(\frac{1}{4} g^2\right) t^4 + (S_z g - M \|\vec{V}\|^2) t^2 + 2M(\vec{S} \cdot \vec{V})t + (S_z^2 - M \|\vec{S}_{xy}\|^2) = 0}
\end{aligned}$$

We now have a quartic polynomial in t containing no other unknowns.
In standard form, the coefficients are as follows:

$$At^4 + Bt^3 + Ct^2 + Dt + E = 0$$

$$\begin{aligned}
A &= \frac{1}{4} g^2 \\
B &= 0 \\
C &= S_z g - M \|\vec{V}\|^2 \\
D &= 2M(\vec{S} \cdot \vec{V}) \\
E &= S_z^2 - M \|\vec{S}_{xy}\|^2 \\
(M &= \tan^2 \theta)
\end{aligned}$$

We can now find the solutions for t and use it to calculate the other unknowns.

2. Find α and u

$\vec{L}$ is now known.

Calculating α

From the common calculations earlier:

$$\boxed{\alpha = \angle \vec{L}}$$

Calculating u

From the common calculations earlier:

$$\boxed{u = \frac{\|\vec{L}\|}{t \cos \theta}}$$

u Parameter Approach

Suppose u is a parameter for the purposes of our calculations.

1. Find t

Vector Kinematics Equation

Start with the kinematics motion equation, for 3D space:

$$\hookrightarrow \vec{S} = \vec{v}t + \frac{1}{2} \vec{g}t^2$$

Rearrange:

$$\begin{aligned}
(\text{substitute for } \vec{v}) &\implies \vec{S} = (u\hat{w} + \vec{V})t + \frac{1}{2}\vec{g}t^2 \\
(\text{distribute } t) &\implies \vec{S} = u\hat{w}t + \vec{V}t + \frac{1}{2}\vec{g}t^2 \\
(\text{rearrange terms}) &\implies \vec{S} - \frac{1}{2}\vec{g}t^2 - \vec{V}t = u\hat{w}t
\end{aligned}$$

Eliminate Other Unknowns α, θ

The only other unknown variable in the equation is $\hat{w}$, which contains both α and θ unknowns. Use vector magnitude to eliminate $\hat{w}$, since $\|\hat{w}\| = 1$:

$$\begin{aligned}
(\text{take magnitude of sides}) &\implies \left\| \vec{S} - \frac{1}{2}\vec{g}t^2 - \vec{V}t \right\| = \|u\hat{w}t\| \\
(\text{right side: pull out scalars}) &\quad \|u\hat{w}t\| = u\|\hat{w}\|t = u(1)t = ut \\
(\text{substitute right side back in}) &\implies \left\| \vec{S} - \frac{1}{2}\vec{g}t^2 - \vec{V}t \right\| = ut
\end{aligned}$$

The result is an equation with only one unknown variable, t .

Convert Equation to Quartic

The left side of the equation has a vector with components and magnitude:

$$\begin{aligned}
(\text{left side vector}) \quad (\vec{S} - \frac{1}{2}\vec{g}t^2 - \vec{V}t) &= \langle S_x - V_x t, S_y - V_y t, S_z + \frac{1}{2}gt^2 \rangle \\
(\text{magnitude formula}) \quad \left\| \vec{S} - \frac{1}{2}\vec{g}t^2 - \vec{V}t \right\| &= \sqrt{(S_x - V_x t)^2 + (S_y - V_y t)^2 + (S_z + \frac{1}{2}gt^2)^2}
\end{aligned}$$

Substitute the formula for the magnitude of the vector in on the left.

$$\begin{aligned}
(\text{substitute magnitude on left}) &\implies \sqrt{(S_x - V_x t)^2 + (S_y - V_y t)^2 + (S_z + \frac{1}{2}gt^2)^2} = ut \\
(\text{square sides}) &\implies (S_x - V_x t)^2 + (S_y - V_y t)^2 + (S_z + \frac{1}{2}gt^2)^2 = (ut)^2
\end{aligned}$$

Expand and rearrange into standard quartic form:

$$\begin{aligned}
(\text{apply squaring}) &\implies (S_x^2 - 2S_x V_x t + V_x^2 t^2) + (S_y^2 - 2S_y V_y t + V_y^2 t^2) + (S_z^2 + S_z g t^2 + \frac{1}{4}g^2 t^4) = u^2 t^2 \\
(\text{rearrange terms by } t \text{ factor}) &\implies \frac{g^2}{4}t^4 + S_z g t^2 + V_x^2 t^2 + V_y^2 t^2 - u^2 t^2 - 2S_x V_x t - 2S_y V_y t + S_x^2 + S_y^2 + S_z^2 = 0 \\
(\text{factor out each } t \text{ power}) &\implies (\frac{g^2}{4})t^4 + (S_z g + V_x^2 + V_y^2 - u^2)t^2 - 2(S_x V_x + S_y V_y)t + (S_x^2 + S_y^2 + S_z^2) = 0
\end{aligned}$$

Finally, simplify with vector properties:

$$\begin{aligned}
V_x^2 + V_y^2 &= \|\vec{V}\|^2 \\
S_x^2 + S_y^2 + S_z^2 &= \|\vec{S}\|^2 \\
S_x V_x + S_y V_y &= S_x V_x + S_y V_y + (S_z \cdot 0) = \vec{S} \cdot \vec{V}
\end{aligned}$$

$$(\text{substitute}) \implies \boxed{\left(\frac{g^2}{4}\right)t^4 + (S_z g + \|\vec{V}\|^2 - u^2)t^2 - 2(\vec{S} \cdot \vec{V})t + \|\vec{S}\|^2 = 0}$$

We now have a quartic polynomial in t containing no other unknowns.
In standard form, the coefficients are as follows:

$$At^4 + Bt^3 + Ct^2 + Dt + E = 0$$

$$A = \frac{1}{4}g^2$$

$$B = 0$$

$$C = S_z g + \|\vec{V}\|^2 - u^2$$

$$D = -2(\vec{S} \cdot \vec{V})$$

$$E = \|\vec{S}\|^2$$

We can now find the solutions for t and use it to calculate the other unknowns.

2. Find α and θ from t

$\vec{L}$ is now known.

Calculating α

From the common calculations earlier:

$$\boxed{\alpha = \angle \vec{L}}$$

Calculating θ

From the common calculations earlier:

$$u = \frac{\|\vec{L}\|}{t \cos \theta}$$

Solve for θ :

$$(\text{multiply sides by } \frac{\cos \theta}{u}) \implies \cos \theta = \frac{\|\vec{L}\|}{ut}$$

$$(\text{take arccos of sides}) \implies \boxed{\theta = \arccos\left(\frac{\|\vec{L}\|}{ut}\right)}$$

(Since θ is physically always in the interval $[0, \frac{\pi}{2}]$, there is no need to find other valid angles.)

Analysis of t Quartic

In both approaches, we find a quartic in t .

For both, $A = \frac{1}{4}g^2$ and $B = 0$.

A quartic with $B = 0$ is called a *depressed quartic*, and it is simpler to solve for its roots.

Note that we are looking for physically possible solutions.

$$t \in \mathbb{R}^+ \text{ (} t \text{ must be real and positive)}$$

For an arbitrary quartic, there may be 0, 2, or 4 real solutions for t .

For the u parameter approach, intuitively there can probably be only either 0 or 2 real positive solutions for t corresponding to a high and low trajectory.

For the θ parameter approach, intuitively there can only be 0 or 1 real positive solutions for t . If there are 0, it is because θ is lower than the target's *elevation angle*, which is equal to $\text{atan2}(S_z, \|\vec{S}_{xy}\|)$.

A quartic always has a closed-form solution. Using Ferrari's method for the quartic along with either Cardano's method or the trigonometric method for the resolvent cubic can yield these real solutions without using complex numbers in any of the steps.

The larger values of t correspond to steeper trajectories (lobs) with a larger θ . These are ideal for the hub.

The smaller values of t correspond to shallow, direct trajectories that may be used for other targets.

An interesting observation of both quartics is that all terms have units of distance squared, under dimensional analysis. This makes sense because the original equation had distance terms, and then one algebra step squared both sides.

Extra Approach for Ignoring z Direction

Sometimes it is desirable to ignore the z direction (all z components) and only consider the horizontal launching direction.

Then, θ only has an impact on the horizontal exit speed, so an arbitrary value should be chosen for it. Usually, it should be as close to 0 as possible to maximize horizontal exit speed.

We need to find α and u , but since they depend on each other, choose a value for u , leaving α as the one unknown.

α can then be calculated without finding t . In fact, the magnitude of the horizontal displacement does not matter, only its direction. This makes sense because α simply has to make the launch direction point towards the target, regardless of the distance.

Problem Description

Given:

$$u, \theta, \vec{V}, \vec{S}_{xy}$$

Find α such that $\vec{v}_{xy}$ and $\vec{S}_{xy}$ are parallel.

Calculations

For conciseness, define:

$$\begin{aligned}\phi &:= \angle \vec{S}_{xy} \\ \sigma &:= u \cos \theta\end{aligned}$$

Then:

$$\begin{aligned}\vec{v}_{xy} &= \langle u \cos \theta \cos \alpha + V_x, u \cos \theta \sin \alpha + V_y \rangle \\ (\text{substitute } \sigma \text{ in}) \implies \vec{v}_{xy} &= \langle \sigma \cos \alpha + V_x, \sigma \sin \alpha + V_y \rangle\end{aligned}$$

$$\begin{aligned}
\vec{S}_{xy} &= \langle S_x, S_y \rangle \\
(\text{vector properties}) &\implies \vec{S}_{xy} = \|\vec{S}_{xy}\| \langle \cos \angle \vec{S}_{xy}, \sin \angle \vec{S}_{xy} \rangle \\
(\text{distribute and substitute } \phi \text{ in}) &\implies \vec{S}_{xy} = \langle \|\vec{S}_{xy}\| \cos \phi, \|\vec{S}_{xy}\| \sin \phi \rangle
\end{aligned}$$

One way to see if two vectors are parallel is to check that their slopes are the same:

$$\begin{aligned}
&\hookrightarrow \frac{v_y}{v_x} = \frac{S_y}{S_x} \\
(\text{substitute for all values}) &\implies \frac{\sigma \sin \alpha + V_y}{\sigma \cos \alpha + V_x} = \frac{\|\vec{S}_{xy}\| \sin \phi}{\|\vec{S}_{xy}\| \cos \phi} = \frac{\sin \phi}{\cos \phi} \\
(\text{cross multiply}) &\implies (\sigma \sin \alpha + V_y) \cos \phi = (\sigma \cos \alpha + V_x) \sin \phi \\
(\text{distribute}) &\implies \sigma \sin \alpha \cos \phi + V_y \cos \phi = \sigma \cos \alpha \sin \phi + V_x \sin \phi \\
(\text{rearrange terms}) &\implies \sigma \sin \alpha \cos \phi - \sigma \cos \alpha \sin \phi = V_x \sin \phi - V_y \cos \phi
\end{aligned}$$

The right side can be simplified using vector cross product:

$$\begin{aligned}
(\text{cross product formula}) \quad &\vec{V} \times \vec{S}_{xy} = V_x S_y - V_y S_x \\
(\text{substitute for } S_x, S_y) &\implies \vec{V} \times \vec{S}_{xy} = V_x (\|\vec{S}_{xy}\| \sin \phi) - V_y (\|\vec{S}_{xy}\| \cos \phi) \\
(\text{divide sides by } \|\vec{S}_{xy}\|) &\implies \frac{\vec{V} \times \vec{S}_{xy}}{\|\vec{S}_{xy}\|} = V_x \sin \phi - V_y \cos \phi
\end{aligned}$$

Substitute the expression into the equation and continue:

$$\begin{aligned}
(\text{substitute}) &\implies \sigma \sin \alpha \cos \phi - \sigma \cos \alpha \sin \phi = \frac{\vec{V} \times \vec{S}_{xy}}{\|\vec{S}_{xy}\|} \\
(\text{divide by } \sigma) &\implies \sin \alpha \cos \phi - \cos \alpha \sin \phi = \frac{\vec{V} \times \vec{S}_{xy}}{\sigma \|\vec{S}_{xy}\|}
\end{aligned}$$

Now, for the left side, use the sine difference identity:

$$\sin(\alpha - \phi) = \sin \alpha \cos \phi - \cos \alpha \sin \phi$$

Substitute and solve for α :

$$\begin{aligned}
(\text{substitute with identity}) &\implies \sin(\alpha - \phi) = \frac{\vec{V} \times \vec{S}_{xy}}{\sigma \|\vec{S}_{xy}\|} \\
(\text{take arcsin of sides}) &\implies \alpha - \phi = \arcsin\left(\frac{\vec{V} \times \vec{S}_{xy}}{\sigma \|\vec{S}_{xy}\|}\right) \\
(\text{add } \phi \text{ to sides}) &\implies \alpha = \phi + \arcsin\left(\frac{\vec{V} \times \vec{S}_{xy}}{\sigma \|\vec{S}_{xy}\|}\right)
\end{aligned}$$

Substitute the expressions for ϕ and σ back in:

$$\alpha = \angle \vec{S}_{xy} + \arcsin\left(\frac{\vec{V} \times \vec{S}_{xy}}{u \cos(\theta) \|\vec{S}_{xy}\|}\right)$$

We will ignore the second branch/solution of arcsin giving a second α (corresponding to a related *supplementary* angle facing in the wrong direction).

A real solution for α exists when the arcsin input is in arcsin's domain (valid inputs) of $[-1, 1]$:

$$\left| \frac{\vec{V} \times \vec{S}_{xy}}{u \cos(\theta) \|\vec{S}_{xy}\|} \right| \leq 1 \iff \alpha \in \mathbb{R}$$

Converting to Robot Coordinate System

Since α and θ are in field coordinate system, they must first be converted to the robot coordinate system in order to be usable by the robot.

Find unknowns α_r, θ_r from known α, θ .

Common Case

If $\text{pitch}(\mathbf{R}_R) = \text{roll}(\mathbf{R}_R) = 0$ (the robot is flat on the ground), then the calculations are trivial.

The yaw is offset by the robot yaw.

The pitch is the same since the robot is flat.

$$\begin{aligned} \hookrightarrow \alpha_r &= \alpha - \text{yaw}(\mathbf{R}_R) \\ \hookrightarrow \theta_r &= \theta \end{aligned}$$

Generalization

To generalize to any arbitrary $\mathbf{R}_R$, use either intuition or rotation algebra.

Define or recall:

$$\begin{aligned} \mathbf{R}_R &:= (\text{robot 3D rotation, FCS}) \\ \hat{w} &:= (\text{unit launch direction}) = \mathbf{R}(0, -\theta, \alpha) \hat{i} \\ \mathbf{R}_S &:= (\text{launcher rotation, RCS}) = \mathbf{R}(0, -\theta_r, \alpha_r) \end{aligned}$$

(Negative pitch tilts upwards for 3D rotations.)

Intuition

We want the launcher to aim in the desired direction of $\hat{w}$. We need to find the rotation $\mathbf{R}_S$ (which is in RCS) from the direction $\hat{w}$ (which is in FCS).

Convert the direction $\hat{w}$ to RCS. (Find $\hat{w}$ relative to the robot rotation.) Call this new direction $\hat{d}$. Now, since they are in the same reference frame of RCS, one can calculate θ_r and α_r from $\hat{d}$.

Rigorous Algebra

The direction of $\mathbf{R}_S$, after being converted to FCS by applying $\mathbf{R}_R$, is $\hat{w}$.

$$\hookrightarrow \mathbf{R}_R \mathbf{R}_S \hat{i} = \hat{w}$$

$$(\text{apply } \mathbf{R}_R^{-1} \text{ to sides}) \implies \mathbf{R}_R^{-1}(\mathbf{R}_R \mathbf{R}_S \hat{i}) = \mathbf{R}_R^{-1}(\hat{w})$$

$$(\text{associative property}) \implies (\mathbf{R}_R^{-1} \mathbf{R}_R) \mathbf{R}_S \hat{i} = \mathbf{R}_R^{-1} \hat{w}$$

$$(\text{substitute } \mathbf{R}_R^{-1} \mathbf{R}_R = \mathbf{I}) \implies (\mathbf{I}) \mathbf{R}_S \hat{i} = \mathbf{R}_R^{-1} \hat{w}$$

$$(\text{applying } \mathbf{I} \text{ has no effect}) \implies \mathbf{R}_S \hat{i} = \mathbf{R}_R^{-1} \hat{w}$$

We find that the direction of $\mathbf{R}_S$ is $\mathbf{R}_R^{-1} \hat{w}$, which is actually just $\hat{w}$ in RCS.

Finding θ_r and α_r from $\hat{d}$

$\hat{d}$ is the desired direction unit vector:

$$\hat{d} := (\hat{w} \text{ in RCS}) = \mathbf{R}_R^{-1} \hat{w}$$

$$\hat{d} = (\text{forward vector of } \mathbf{R}_S)$$

Simply use the components of $\hat{d}$ to find θ_r and α_r , keeping roll($\mathbf{R}_S$) = 0.

$$\hookrightarrow -\theta_r = \text{pitch}(\mathbf{R}_S) = \arcsin\left(\frac{d_z}{\|\hat{d}\|}\right)$$

$$\hookrightarrow \alpha_r = \text{yaw}(\mathbf{R}_S) = \angle \vec{d}_{xy}$$

$$\implies \boxed{\theta_r = -\arcsin(d_z)}$$

$$\implies \boxed{\alpha_r = \text{atan2}(d_y, d_x)}$$

Solution

Summary of findings, without intermediate calculations.

Variables

Given:

$$\vec{P}_{xyz}, \vec{V}_{xy}, \mathbf{R}_R, \vec{T}_{xyz}, \vec{Q}_{xyz}, g$$

Find values for:

$$\alpha, \theta, u$$

$$\alpha_r, \theta_r$$

For convenience, define:

$$\vec{S}_{xyz} = \vec{T} - (\vec{P} + \mathbf{R}_R \vec{Q})$$

Now, choose either u or θ to make a parameter, and give it a guessed or set value.

Find t

Solve a depressed quartic for time t :

$$At^4 + Ct^2 + Dt + E = 0$$

$$A = \frac{1}{4}g^2$$

Coefficients if θ is a parameter:

$$C = S_z g - M \|\vec{V}\|^2$$

$$D = 2M(\vec{S} \cdot \vec{V})$$

$$E = S_z^2 - M \|\vec{S}_{xy}\|^2$$

$$(M = \tan^2 \theta)$$

Coefficients if u is a parameter:

$$C = S_z g + \|\vec{V}\|^2 - u^2$$

$$D = -2(\vec{S} \cdot \vec{V})$$

$$E = \|\vec{S}\|^2$$

t must be real and positive.

$$t \in \mathbb{R}^+$$

Choose a t based on its size, with larger t having a steeper trajectory.

(If there is no valid t , try another θ or approach.)

Find Unknowns

For convenience, define:

$$\vec{L}_{xy} := \vec{S}_{xy} - \vec{V}t$$

Find α :

$$\alpha = \angle \vec{L}$$

If θ is a parameter, find u :

$$u = \frac{\|\vec{L}\|}{t \cos \theta}$$

If u is a parameter, find θ :

$$\theta = \arccos\left(\frac{\|\vec{L}\|}{ut}\right)$$

Ignoring z Direction

If the z component of the target position is not important, then choose any θ and u .

Then, find α :

$$\alpha = \angle \vec{S}_{xy} + \arcsin\left(\frac{\vec{V} \times \vec{S}_{xy}}{u \cos(\theta) \|\vec{S}_{xy}\|}\right)$$

Convert θ and α to RCS

If robot is not tilted:

$$\alpha_r = \alpha - \text{yaw}(\mathbf{R}_R)$$

$$\theta_r = \theta$$

$$(\text{pitch}(\mathbf{R}_R) = \text{roll}(\mathbf{R}_R) = 0)$$

In general, for any robot rotation:

$$\hat{w} = \langle \cos \theta \cos \alpha, \cos \theta \sin \alpha, \sin \theta \rangle$$

$$\hat{d} := \mathbf{R}_R^{-1} \hat{w} = (\hat{w} \text{ in RCS})$$

$$\theta_r = -\arcsin(d_z)$$

$$\alpha_r = \text{atan2}(d_y, d_x)$$

Physical Considerations

Extra Constraints

u Limit

There is a maximum mechanically achievable ω , so there is an upper limit on u (that varies based on θ).

This limit makes some long trajectories or very direct trajectories invalid.

θ_r Limits

There are important mechanical limits on how low and high the launcher pitch can go.

Due to this, some high trajectories will be unachievable. Also, maximal horizontal launching is limited with a minimum pitch.

α_r Constraints

The turret has a maximum and minimum achievable yaw. So, the turret may need to rotate to the other side of the limit to continue following a changing desired rotation.

Additionally, there may be areas where launching isn't possible because of other mechanisms on the robot that block it.

t Constraints

There may be an upper limit on t for the fuel to actually score points, due to hub deactivation.

There may also be a lower limit if the robot is launching fuel early, before the hub is activated.

Obstacles

Some trajectories are physically impossible because of obstacles.

Primarily, the hub walls require steep trajectories.

One simple way to validate the trajectory is to check that z component of the final velocity is sufficiently negative.

A more detailed method is to check that at the time when the projectile is right above the obstacle, the projectile has enough height.

Sources of Error

This is a list of possible sources of error, sorted by most significant to least significant.

The launch speed of the projectile likely varies due to natural variance in the projectile and flywheel speed. Launching many projectiles quickly may affect the flywheel speed, but this has to be tested. The variability of projectile speeds should be determined. The error likely increases with higher launch speeds. This is a large source of error.

Projectile air resistance (drag) is likely a large source of error, causing undershooting. The error increases drastically with launch speed. However, research needs to be done to examine its impact.

The magnus effect for backspin applies since the projectile is given backspin by the launching mechanism. This generates lift, which causes overshooting. The error increases for greater backspin. It is unknown how significant this source of error is.

Error in data such as robot pose and robot velocity yield proportional launching error. But robot odometry and vision should be able to keep the accuracy of these within suitable error bounds for launching.

Currently, the rotational velocity of the robot is not considered. This may cause a lot of error at high spin rates.

The true launch angle in both pitch and yaw may slightly vary.

The delay for the fuel to actual exit the launcher may give some error.

It is expected that the launcher can always match the desired θ_r and α_r with negligible latency.

The muzzle point changes when launcher pitch changes, but this error is always only a couple of inches and so is negligible. The launcher position $\vec{Q}$ should be chosen well to reduce this error.

The velocity from the launcher rotating and aiming likely has negligible impact on the projectile's velocity, but this should be tested. The error increases with faster launcher rotation.

Most of the time, $V_z = 0$. But in any case, it should be completely negligible. The tower climb is likely slow. The hub cannot be targeted from the bump.

Reducing and Compensating for Error

Having a greater θ (steeper trajectory) makes the error have a lesser impact on whether the fuel makes it into the hub.

Reducing exit speed reduces many sources of error.

Reducing θ counteracts drag.

Algorithm

These approaches must be chosen and used by the robot program, possibly multiple times, to find an adequate trajectory.

The parameter values necessary for each approach also need to be decided.

The goal of the trajectory:

- Make it to or into the target (success)

- Reduce expected error
- Minimize the impact of error on success
- Satisfy constraints

Approach Analysis

θ Parameter Approach

The main pro is that there are obvious choices for θ :

- The maximum θ achievable (assuming under 80 degrees)
- The θ that minimizes u for the trajectory
- The elevation angle of the target
- The minimum θ achievable (assuming above 0 degrees)

The maximum θ is a great or ideal trajectory for a hub target because steep trajectories are more accurate and consistent, since trajectory error causes less error in horizontal distance. However, this trajectory also has the longest duration.

The θ that minimizes u is given by:

$$\theta = \frac{1}{2} \left(\frac{\pi}{2} + \text{atan2}(S_z, \|\vec{S}_{xy}\|) \right)$$

The proof of this requires calculus and is left as an exercise to the reader.

This θ is useful because it is the theoretical ‘safest’ or ‘most valid’ trajectory; if there are any valid trajectories (launching the target is achievable), then the trajectory for this θ is valid too because it only requires the minimum u .

The effect of θ or changes to it on the trajectory are also clear and predictable. Increasing θ will lead to a higher trajectory and a longer duration.

However, this approach heavily relies on an accurate model of the exit speed function $U(\omega, \theta)$. Error or variation can propagate significantly.

u Parameter Approach

This approach is probably not viable to use because it has too many issues.

In order to keep u a constant, ω must be changed to counteract the effect of the calculated θ .

There is also not an obvious u to choose. There is no one maximum u due to the dependence on θ .

Additionally, the trajectory generated from a chosen u is difficult to predict, so finding a good and valid trajectory is harder. For example, choosing a too large u may result in an unachievably high θ , while choosing a too small u may result in no solution.

Ignore z Approach

Ignoring the z direction can be useful when launching fuel towards or out of a zone.

Choose the minimum possible θ . Then, choose any arbitrary ω such as the maximum.

However, it should be made sure that if the trajectory is intended to bounce off of a wall, that it does not fly over it instead.

Strategy

If the height of the target does not matter, then use the approach ignoring z .

If this trajectory intends to rebound off of a wall, check that the projectile will not go over it.

Otherwise, repeatedly use the θ parameter approach to find a solution using the bisection algorithm.

Trajectory Constraints

The set of constraints determine whether a guessed θ is valid, and whether any valid θ must be higher or lower than the guess.

All constraints must be satisfied for a θ to be valid.

A *lower* constraint puts a lower bound on the valid range of θ . If a given θ fails to satisfy a lower constraint, then a valid θ must be higher than the guess.

Similarly, an *upper* constraint puts an upper bound on the valid range of θ . If the constraint is not satisfied, a valid θ must be lower than the guess.

If *any* θ fails both a lower constraint and an upper constraint, then there is no valid solution for θ .

A “soft” constraint is one that does not have to be satisfied for a trajectory to be valid, but makes certain trajectories preferable or better than others.

Additionally, these soft constraints are more abstract. They are not just either pass or fail, but more like a score or rating.

A soft upper constraint makes shallower trajectories more preferable, while a soft lower constraint makes steeper ones better.

(If a constraint is not mentioned to be soft, assume it is a “hard” constraint.)

Possible Constraints The maximum ω (and, by extension, $\max u$) is an extremely important constraint. If θ is greater than the θ that minimizes u , then this is an upper constraint. If θ is less than it, then this is a lower constraint. If θ is equal (or approximately equal) to it, then it is both an upper and lower constraint. A minimum u may similarly be a constraint if for some reason desirable.

Obstacles that must be launched over such as hub walls are a lower constraint because steeper trajectories are higher than shallower trajectories at all parts of the arc.

Obstacles that must be launched under such as the ceiling are an upper constraint.

A maximum time of flight is an upper constraint because steeper trajectories always take longer.

A minimum time of flight is a lower constraint.

Mechanical minimum and maximum limits on θ are lower and upper constraints.

If the algorithm is implemented well, then none of the guessed θ values will fail these constraints, making them skippable.

Possible Soft Constraints Minimizing possible horizontal error of the trajectory endpoint is a soft lower constraint. Steeper trajectories have a slower horizontal speed and so are less prone to horizontal error compared to shallower trajectories that may greatly vary in horizontal displacement due to error. Similarly, minimizing possible vertical error is a soft upper constraint.

Minimizing time of flight is a soft upper constraint because steeper trajectories have a longer duration.

Similarly, maximizing time of flight is a soft lower constraint.

Reducing u and ω is a soft upper constraint when θ is greater than the θ that minimizes u , and a soft lower constraint when θ is less than it.

Reducing u probably helps improve consistency for u , as the magnitude of variation scales with the expected u .

Bisection Algorithm

First, decide whether θ should be maximized or minimized based on the soft constraints.

Then, initialize and track a *low* θ , a *high* θ , and a *guess* θ as follows:

Set the high θ to the maximum possible θ below 90 degrees.

If θ is being maximized, calculate the trajectory; if it is valid, use that as the final θ , but if it fails a lower constraint, then there is no valid solution for θ .

Set the low θ to the elevation angle of the target if that is greater than the minimum possible θ .
(This θ is always invalid.)

Otherwise, if the minimum possible θ is greater than the elevation angle, use that for the low θ instead.
If θ is being minimized, calculate the trajectory; if it is valid, use that as the final θ , but if it fails an upper constraint, then there is no valid solution for θ .

Set the guess θ to the θ that maximizes u , which is the average of the elevation angle and 90 degrees.

Then, for a set amount of iterations or until it is determined there is no solution, repeat the following steps:

1. Calculate the trajectory of the current guess θ
2. If it fails both a lower constraint and an upper constraint, then there is no solution for θ
3. If it fails a lower constraint, or if it is valid and θ should be maximized, set the low θ to the guess θ
4. If it fails an upper constraint, or if it is valid and θ should be minimized, set the upper θ to the guess θ
5. Set the guess θ to the average of the new low θ and high θ for the next iteration

During this, keep track of the highest valid θ if θ is being maximized, or the lowest valid one if it is being minimized. The end result will be the final θ .

If no valid θ was found within the amount of iterations, then there may or may not have been valid solutions that were not found.

Notes Each extra iteration cuts the maximum possible error from the optimal trajectory in half.

The final θ from the previous calculation may possibly be utilized to help find the current final θ , but it is simpler and more robust to just use more iterations.

This bisection algorithm does not have infinite precision, so there are only a finite number of possible values for θ it can give.

Because of this, it may possibly cause minor back and forth jittering as parameters vary slightly and shift the amount of error.

The actual used θ may be adjusted to account for biased error, such as reducing θ by a few degrees to counteract drag.

Conclusion

In order to aim the launcher of a moving robot to shoot a target, an analytic approach for calculating an exact trajectory from a given launch angle can be combined with a computational solver for guessing and optimizing the launch angle under a set of physical constraints.

The main hurdle for accuracy now becomes to reduce the error in the actual trajectory due to physical factors.

Many teams with a turret launcher might instead focus more on computational solving to approach this problem, but an analytical approach guarantees no numeric error for a chosen trajectory, along with perfectly handling any arbitrary robot velocity.

These trajectory calculations enhance the effectiveness of the robot without any mechanical improvements, which is a goal of the programming subteam.

Attribution

This file was written by Anthony Le, @aatle on GitHub.

Some techniques used in the calculations for the u parameter approach and ignoring z direction approach were found with the help of an AI, GPT-5.
None of this file was written with AI.

License

REBUILT Trajectory Calculations © 2026 by Anthony Le is licensed under Creative Commons Attribution 4.0 International. To view a copy of this license, visit <https://creativecommons.org/licenses/by/4.0/>